



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 7

NOMBRE COMPLETO: Ricardo Emmanuel Galicia Tequianes

N° de Cuenta: 118001740

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 10 de octubre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Práctica 7: Iluminación 1

La escena parte del código base de la práctica 7 con el modelo de Phong activado en el fragment shader. Los tres ejercicios del reporte se hicieron con mis modelos con jerarquía: el rover de la práctica 5 lleva un faro frontal, y el avión de la práctica 6 lleva una luz hacia abajo y un faro hacia adelante. También está la lámpara importada con su luz puntual blanca que pide el manual. El proyecto se compiló con CMake y se ejecutó en macOS (OpenGL 4.1 Metal, GLSL 4.10).

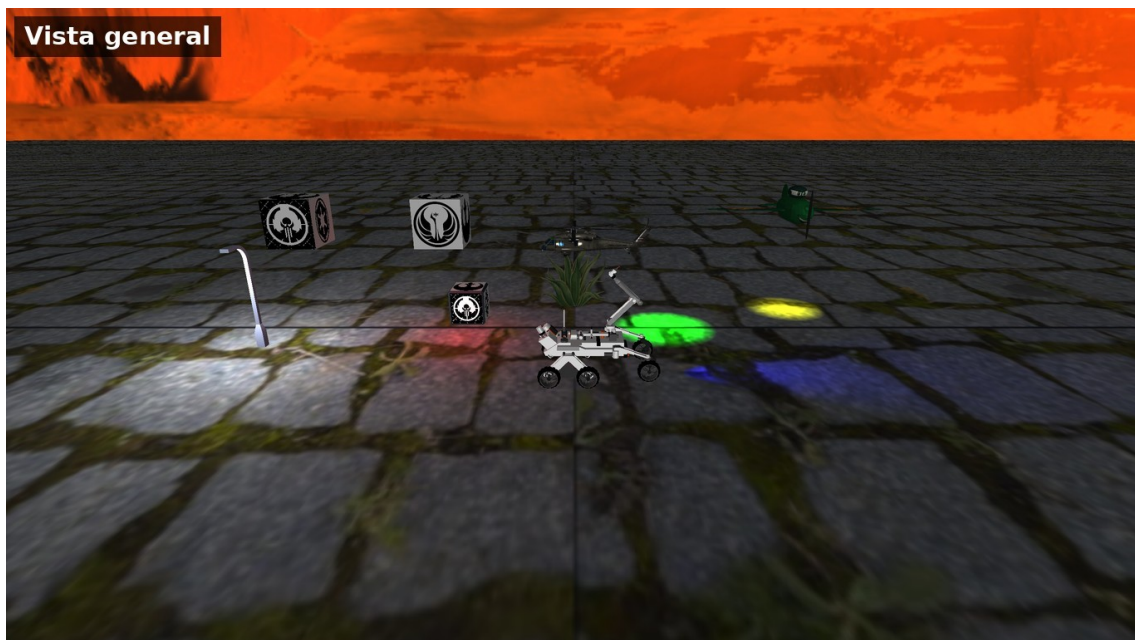


Figura 1. Escena completa con la linterna apagada.

1.- Ejecución de los ejercicios.

Las tres luces del reporte siguen la misma idea: la posición de cada spotlight se calcula en cada cuadro con la matriz que se usa para dibujar el modelo. Así la luz queda unida al modelo en su jerarquía y se mueve con él sin código extra para cada tecla. Las teclas se leen en el ciclo principal con el arreglo keys de Window y el desplazamiento se multiplica por deltaTime.

```

649         //Rover: Y hacia adelante (+X, donde está su frente), U hacia atrás
650         if (keys[GLFW_KEY_Y]) posRoverX += VELOCIDAD_ROVER * deltaTime;
651         if (keys[GLFW_KEY_U]) posRoverX -= VELOCIDAD_ROVER * deltaTime;
652         posRoverX = glm::clamp(posRoverX, -15.0f, 15.0f);
653         //Avión: flecha arriba sube, flecha abajo baja
654         if (keys[GLFW_KEY_UP]) posAvionY += VELOCIDAD_AVION * deltaTime;
655         if (keys[GLFW_KEY_DOWN]) posAvionY -= VELOCIDAD_AVION * deltaTime;
656         posAvionY = glm::clamp(posAvionY, 1.0f, 14.0f);
657         //Avión: H hacia adelante (+Z), J hacia atrás
658         if (keys[GLFW_KEY_H]) posAvionZ += VELOCIDAD_AVION * deltaTime;
659         if (keys[GLFW_KEY_J]) posAvionZ -= VELOCIDAD_AVION * deltaTime;
660         posAvionZ = glm::clamp(posAvionZ, -15.0f, 12.0f);
661         //L enciende o apaga la linterna (solo al presionar, no mientras se mantiene)
662         if (keys[GLFW_KEY_L] && !teclaLAnterior) linternaEncendida = !linternaEncendida;
663         teclaLAnterior = keys[GLFW_KEY_L];
664     }

```

Figura 2. Teclas del rover (Y/U), del avión (flechas y H/J) y de la linterna (L).

```

576     //luz del avión: amarilla, en la parte inferior del fuselaje y apuntando al piso;
577     //su posición se actualiza en cada cuadro con la altura del avión
578     spotLights[2] = SpotLight(1.0f, 1.0f, 0.0f,
579         0.0f, 3.0f,
580         0.0f, 0.0f, 0.0f,
581         0.0f, -1.0f, 0.0f,
582         1.0f, 0.1f, 0.01f,
583         25.0f);
584     spotLightCount++;
585
586     //faro del rover: azul, en el frente del cuerpo y apuntando hacia adelante (+X), un poco al piso;
587     //su posición se actualiza en cada cuadro con la posición del rover
588     spotLights[3] = SpotLight(0.0f, 0.0f, 1.0f,
589         0.0f, 5.0f,
590         0.0f, 0.0f, 0.0f,
591         1.0f, -0.2f, 0.0f,
592         1.0f, 0.05f, 0.005f,
593         25.0f);
594     spotLightCount++;
595
596     //faro delantero del avión: naranja, delante de la hélice y apuntando hacia adelante (+Z),
597     //un poco hacia el piso; se mueve con el avión al avanzar, retroceder, subir o bajar
598     spotLights[4] = SpotLight(1.0f, 0.5f, 0.0f,
599         0.0f, 5.0f,
600         0.0f, 0.0f, 0.0f,
601         0.0f, -0.5f, 1.0f,
602         1.0f, 0.05f, 0.005f,
603         20.0f);
604     spotLightCount++;

```

Figura 3. Declaración de las tres spotlight: luz inferior del avión, faro del rover y faro delantero del avión.

```

691     //Matrices base del rover y del avión: se usan para dibujarlos y para colocar sus luces,
692     //así la luz siempre queda en el mismo punto del modelo aunque este se mueva
693     glm::mat4 baseRover(1.0);
694     baseRover = glm::translate(baseRover, glm::vec3(posRoverX, -1.0f, 6.0f));
695     baseRover = glm::scale(baseRover, glm::vec3(ESCALA_ROVER, ESCALA_ROVER, ESCALA_ROVER));
696     glm::vec3 posFaroRover = glm::vec3(baseRover * glm::vec4(6.9f, 2.3f, 0.0f, 1.0f)); //frente del cuerpo
697     spotLights[3].SetPos(posFaroRover);
698
699     glm::mat4 baseAvion(1.0);
700     baseAvion = glm::translate(baseAvion, glm::vec3(13.0f, posAvionY, posAvionZ));
701     baseAvion = glm::scale(baseAvion, glm::vec3(0.75f, 0.75f, 0.75f));
702     glm::vec3 posLuzAvion = glm::vec3(baseAvion * glm::vec4(0.0f, -1.05f, 0.5f, 1.0f)); //debajo del fuselaje
703     spotLights[2].SetPos(posLuzAvion);
704     glm::vec3 posFaroAvion = glm::vec3(baseAvion * glm::vec4(0.0f, 0.0f, 3.4f, 1.0f)); //delante de la hélice
705     spotLights[4].SetPos(posFaroAvion);
706
707     //información al shader de fuentes de iluminación
708     shaderList[0].SetDirectionalLight(&mainLight);
709     shaderList[0].SetPointLights(pointLights, pointLightCount);
710     //si la linterna está apagada se envía el arreglo a partir de la segunda luz
711     if (!linternaEncendida)
712         shaderList[0].SetSpotLights(spotLights, spotLightCount);
713     else
714         shaderList[0].SetSpotLights(spotLights + 1, spotLightCount - 1);

```

Figura 4. Cada luz se coloca con la matriz de su modelo antes de mandarla al shader.

a) Faro frontal del rover

El rover es el de la práctica 5. Para que también tuviera jerarquía de llantas separé en Blender cada llanta de su pata: las llantas tienen su propio material, así que se separaron por material, se les puso el origen en el centro de su caja envolvente y se exportaron centradas; la pata se exportó con sus coordenadas originales para no mover su punto de unión con el cuerpo. La jerarquía queda cuerpo → pata → llanta, y cada llanta gira sobre su eje Z según la distancia recorrida entre su radio (1.241 por la escala de 0.4).

```
38     for p in piezas:
39         pata = importar("PataRueda_" + p)
40         seleccionar(pata)
41         bpy.ops.object.mode_set(mode='EDIT')
42         bpy.ops.mesh.select_all(action='SELECT')
43         bpy.ops.mesh.separate(type='MATERIAL')
44         bpy.ops.object.mode_set(mode='OBJECT')
45         partes = [o for o in bpy.context.selected_objects]
46         llanta = next(o for o in partes if any("wheels" in m.name for m in o.data.materials))
47         resto = [o for o in partes if o is not llanta]
48         seleccionar(resto[0])
49         for o in resto: o.select_set(True)
50         bpy.ops.object.join()
51         pata = bpy.context.view_layer.objects.active
52         pata.name = "Pata_" + p
53         llanta.name = "Llanta_" + p
54         seleccionar(llanta)
55         bpy.ops.object.origin_set(type='ORIGIN_GEOMETRY', center='BOUNDS')
56         c = llanta.location.copy()
57         # mismo punto en ejes del OBJ (Y arriba): (x, z, -y)
58         info.write(f"{p} {c.x:.4f} {c.z:.4f} {-c.y:.4f}\n")
59         exportar(pata, "Pata_" + p)
60         llanta.location = (0, 0, 0)
61         exportar(llanta, "Llanta_" + p)
62         llanta.location = c
63         # jerarquía en el .blend (después de exportar y sin mover nada): Cuerpo -> Pata -> Llanta
64         for hijo, padre in ((pata, cuerpo), (llanta, pata)):
65             mundo = hijo.matrix_world.copy()
66             hijo.parent = padre
67             hijo.matrix_world = mundo
```

Figura 5. Parte del script de Blender que separa la llanta, le pone el origen en su centro y exporta las dos piezas.

```
742     //Rover de la práctica 5 con jerarquía: cuerpo -> pata -> llanta
743     //Giro de las llantas = distancia recorrida / radio; negativo porque avanza hacia +X
744     //y la llanta gira sobre su eje Z
745     float giroLlantas = -(posRoverX - INICIO_ROVER_X) / RADIO_LLANTA;
746     model = baseRover;
747     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
748     Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
749     Cuerpo.M.RenderModel(uniformColor);
750     Brazo.M.RenderModel(uniformColor);
751     for (int i = 0; i < 6; i++)
752     {
753         //pata: gira sobre su punto de unión con el cuerpo (en la práctica 5 se movía con teclado)
754         modelaux = model;
755         modelaux = glm::translate(modelaux, hingePata[i]);
756         modelaux = glm::rotate(modelaux, 0.0f, glm::vec3(0.0f, 0.0f, 1.0f));
757         modelaux = glm::translate(modelaux, -hingePata[i]);
758         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
759         Pata_M[i].RenderModel(uniformColor);
760         //llanta: hija de la pata, se lleva a su centro y ahí gira
761         modelaux = glm::translate(modelaux, centroLlanta[i]);
762         modelaux = glm::rotate(modelaux, giroLlantas, glm::vec3(0.0f, 0.0f, 1.0f));
763         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
764         Llanta_M[i].RenderModel(uniformColor);
765     }
```

Figura 6. Dibujo del rover: cada llanta toma la matriz de su pata, se traslada a su centro y gira sobre Z.

El rover avanza con Y y retrocede con U. El faro es spotLights[3]: azul (0, 0, 1), intensidad difusa 5, atenuación con 1, lin 0.05 y exp 0.005, cono de 25°. Se coloca en el punto (6.9, 2.3, 0) de la matriz del rover, en el frente del cuerpo, y apunta en (1, -0.2, 0), hacia adelante y un poco al piso. Al avanzar o retroceder, el círculo azul se queda delante del rover.

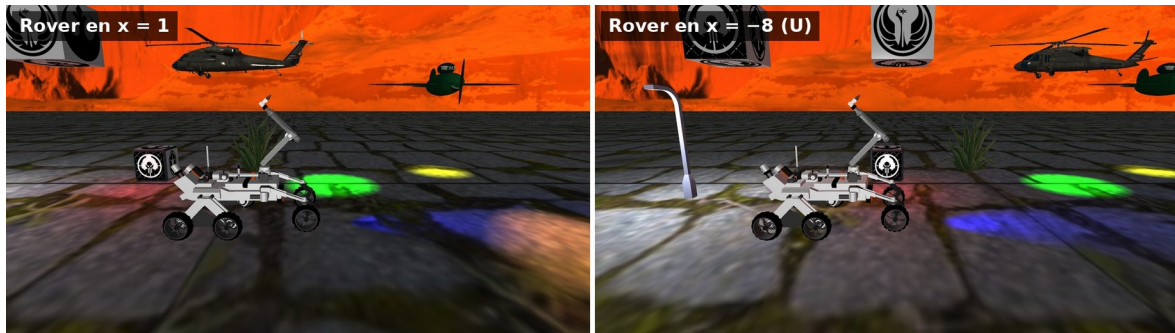


Figura 7. El faro azul se queda delante del rover en las dos posiciones.



Figura 8. La misma llanta antes y después de avanzar 0.35 unidades.

b) Luz del avión hacia abajo al subir y bajar

El avión es el de la práctica 6. spotLights[2] es amarilla (1, 1, 0), intensidad difusa 3, atenuación con 1, lin 0.1 y exp 0.01, cono de 25° y dirección fija (0, -1, 0). Su posición sale de la matriz del avión en el punto (0, -1.05, 0.5), debajo del fuselaje. Con flecha arriba el avión sube y con flecha abajo baja, y la luz va con él.

Con el avión arriba el círculo es más grande y más tenue: el cono abarca más piso y la atenuación crece con la distancia. Con el avión abajo el círculo es pequeño y brillante.

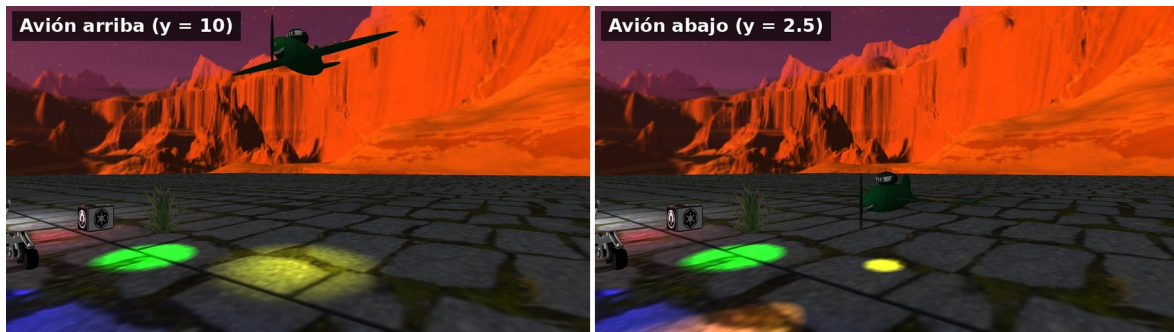


Figura 9. La luz amarilla con el avión a dos alturas distintas.



Figura 10. Altura inicial del avión y vista cercana.

c) Faro delantero del avión al avanzar y retroceder

La hélice del avión está hacia +Z, así que ese es su frente. Con H avanza y con J retrocede; posAvionZ entra en la traslación de la misma matriz base que ya usaba la luz inferior. El faro es spotLights[4]: naranja (1, 0.5, 0), intensidad difusa 5, atenuación con 1, lin 0.05 y exp 0.005, cono de 20°. Se coloca en (0, 0, 3.4), justo delante de la hélice, y apunta en (0, -0.5, 1) para que el círculo caiga en el piso delante del avión. Para tener cinco spotlight subí MAX_SPOT_LIGHTS de 4 a 5 en CommonValues.h y en shader_light.frag, porque los dos valores tienen que coincidir.



Figura 11. Vista lateral: al avanzar el avión con H, el círculo naranja y el amarillo se mueven con él.



Figura 12. Desde atrás del avión se ve el faro naranja delante y la luz amarilla debajo.

d) Lámpara importada con luz puntual blanca

La lámpara es el modelo light-curved del paquete City Kit (Roads) de Kenney, con licencia CC0. Se carga con la clase Model; su textura se llama colormap.png, así que la renombré a colormap_ciudad.png y cambié la referencia en el .mtl. Está en $(-16, -1, 2.6)$, escalada 8 veces y girada 180° . La luz es pointLights[1]: blanca $(1, 1, 1)$, intensidad difusa 2, atenuación con 1, lin 0.09 y exp 0.02, en el punto $(0, 0.62, -0.175)$ de la matriz de la lámpara, justo debajo de la pantalla.

```

544 //Lámpara: se para junto al carril del rover, girada 180° para que su brazo quede sobre el carril.
545 //La luz puntual blanca va justo debajo de la pantalla de la lámpara.
546 glm::mat4 modeloLampara(1.0f);
547 modeloLampara = glm::translate(modeloLampara, glm::vec3(-16.0f, -1.0f, 2.6f));
548 modeloLampara = glm::rotate(modeloLampara, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
549 modeloLampara = glm::scale(modeloLampara, glm::vec3(8.0f, 8.0f, 8.0f));
550 glm::vec3 posFocoLampara = glm::vec3(modeloLampara * glm::vec4(0.0f, 0.62f, -0.175f, 1.0f));
551 pointLights[1] = PointLight(1.0f, 1.0f, 1.0f,
552     0.0f, 2.0f,
553     posFocoLampara.x, posFocoLampara.y, posFocoLampara.z,
554     1.0f, 0.09f, 0.02f);
555 pointLightCount++;

```

Figura 13. Matriz de la lámpara y su luz puntual blanca.



Figura 14. El piso alrededor de la lámpara recibe la luz blanca mientras el rover pasa junto a ella.

Controles: W, A, S, D y ratón mueven la cámara; Y y U, el rover; flecha arriba y flecha abajo suben y bajan el avión; H y J lo mueven hacia adelante y hacia atrás; L enciende o apaga la linterna; Esc cierra el programa.

2.- Problemas y cómo se resolvieron.

Escena sin iluminación en macOS. `glValidateProgram` fallaba con «No vertex array object bound» y `Shader_light.cpp` hacía `return` antes de pedir las ubicaciones de los uniform, así que ninguna luz llegaba al shader. Dejé el mensaje como aviso y quité el `return`.

```

74 //En macOS la validación falla si todavía no hay un VAO enlazado ("No vertex array object bound").
75 //No es un error del shader, así que solo se informa y se continúa para obtener las ubicaciones uniform.
76 glValidateProgram(shaderID);
77 glGetProgramiv(shaderID, GL_VALIDATE_STATUS, &result);
78 if (!result)
79 {
80     glGetProgramInfoLog(shaderID, sizeof(eLog), NULL, eLog);
81     printf("Aviso al validar el programa: '%s'\n", eLog);
82 }

```

Figura 15. La validación ya no detiene la carga de los uniform.

Compilación en la Mac. Los `#include` con diagonal invertida (`gtc\matrix_transform.hpp`) no compilan con clang y se cambiaron por diagonal normal. `practica7.cpp` incluía `Sphere.h`, que no venía en la carpeta y no se usa, así que quité ese include. También agregué un `CMakeLists.txt` que copia shaders, Models y Textures junto al ejecutable.

Contexto de OpenGL. `Window.cpp` pedía OpenGL 4.3 y macOS solo da hasta 4.1 en perfil Core. Se pidió 3.3, que es lo que usan los shaders (`#version 330`).

Cámara girando sola al iniciar. El constructor Window(ancho, alto) no inicializaba mouseFirstMoved, lastX ni lastY. Los inicialicé.

Más de cuatro spotlight. Al agregar el faro del avión el arreglo de luces ya tenía cuatro (linterna, luz verde, avión y rover). Hay que subir el máximo en CommonValues.h y en la constante del fragment shader al mismo tiempo; si solo se cambia uno, el programa manda luces que el shader no tiene o se pierden ubicaciones uniform.

Normales. En CreateObjects, calcAverageNormals se llamaba después de crear las mallas y la pirámide y el agave llegaban a la GPU con normales en cero; moví las llamadas antes. Mi dado de la práctica 6 tenía las normales hacia afuera, y con este shader cada cara se iluminaba solo cuando la luz estaba detrás; se invirtió el signo en AgregarCara y en CrearHolocron.

Patas giradas al exportar desde Blender. El importador de OBJ deja una rotación de 90° en cada objeto. Al emparentar las patas con el cuerpo antes de exportarlas, heredaban esa rotación y el OBJ salía girado sobre X. Exporté primero cada pieza y armé la jerarquía después, conservando su posición.

Textura faltante del rover. El material del brazo apunta a SpacialRoverT301Texture.png, que no está. LoadTextureA ahora regresa false cuando no encuentra la imagen y Model usa plain.png en su lugar, en vez de dejar una textura vacía.

3.- Conclusión:

- a. Los ejercicios del reporte: complejidad, explicación.

La complejidad fue media. Las tres spotlight se resuelven igual: la posición de la luz se calcula con la matriz que se usa para dibujar el modelo, así que el faro del rover y las dos luces del avión no se separan aunque cambien la posición o la escala. La luz puntual de la lámpara es fija y solo hubo que elegir el punto debajo de la pantalla. Lo que más trabajo dio fue la jerarquía de llantas del rover, porque hubo que separar las piezas en Blender sin mover la bisagra de cada pata y calcular el giro a partir de la distancia recorrida.

- b. Comentarios generales.

Faltó explicar en clase hacia dónde deben ir las normales con este shader; se entiende revisando CalcLightByDirection y viendo que el piso usa (0, -1, 0). Sugiero que el código base traiga la línea de iluminación del fragment shader ya activa y con un comentario sobre esa convención.

- c. Conclusión.

Con las tres fuentes del modelo de Phong se ve qué controla cada parámetro: el color y la intensidad difusa definen cuánto ilumina la luz, la atenuación (con, lin, exp) define qué tan rápido se apaga con la distancia y el ángulo de la spotlight define el tamaño del círculo. Poner las luces en la jerarquía de los modelos fue lo que permitió que se movieran con ellos sin código extra para cada tecla.

1. Bibliografía en formato APA

Blender Foundation. (s. f.). *Wavefront OBJ*. Blender Manual.

https://docs.blender.org/manual/en/latest/files/import_export/obj.html

de Vries, J. (s. f.). *Basic lighting*. LearnOpenGL.

<https://learnopengl.com/Lighting/Basic-Lighting>

de Vries, J. (s. f.). *Light casters*. LearnOpenGL.

<https://learnopengl.com/Lighting/Light-casters>

Kenney. (2026). *City Kit (Roads)* [Modelos 3D, licencia CC0].

<https://kenney.nl/assets/city-kit-roads>

Roque RG, J. (2026). *Manual de prácticas: Práctica 7. Iluminación 1* [Material de clase]. Laboratorio de Computación Gráfica e Interacción Humano Computadora, Facultad de Ingeniería, UNAM.

The Khronos Group. (s. f.). *glValidateProgram*. OpenGL 4 Reference Pages.

<https://registry.khronos.org/OpenGL-Refpages/gl4/html/glValidateProgram.xhtml>